

**VIETNAM NATIONAL UNIVERSITY  
UNIVERSITY OF SCIENCE**

**Project 2**

**Building a Serverless Web Application with Zero-Cost &  
Secure Architecture**

**Course Code: CSC11006**

**Student:**

23127158 - Nguyen Thanh Gia Bao

23127404 - Le Tuan Loc

23127438 - Dang Truong Nguyen

**Academic Supervisors:**

Le Ngoc Son

Chung Thuy Linh

**Semester: 2**

**Academic Year: 2025 - 2026**

**Ho Chi Minh City, May 1, 2026**

# TABLE OF CONTENTS

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>TABLE OF CONTENTS</b>                                  | <b>1</b>  |
| <b>LIST OF FIGURES</b>                                    | <b>4</b>  |
| <b>1 Web Application</b>                                  | <b>6</b>  |
| 1.1 Task Assignment and Completion . . . . .              | 6         |
| 1.2 Frontend accessible via the CloudFront URL . . . . .  | 6         |
| 1.3 Backend API . . . . .                                 | 6         |
| 1.3.1 API GET /tasks testing with Postman . . . . .       | 7         |
| 1.3.2 API POST /tasks testing with Postman . . . . .      | 7         |
| 1.3.3 API PUT /tasks/id testing with Postman . . . . .    | 8         |
| 1.3.4 API DELETE /tasks/id testing with Postman . . . . . | 8         |
| 1.4 Database Item . . . . .                               | 9         |
| <b>2 Architecture Diagram</b>                             | <b>10</b> |
| 2.1 Architecture Diagram . . . . .                        | 10        |
| <b>3 Security Evidence Package</b>                        | <b>11</b> |
| 3.1 Frontend Security Evidence . . . . .                  | 11        |
| 3.1.1 SE-1: S3 Block Public Access enabled . . . . .      | 11        |
| 3.1.2 SE-2: Direct S3 access rejected . . . . .           | 12        |
| 3.1.3 SE-3: CloudFront access succeeds . . . . .          | 12        |
| 3.1.4 SE-4: OAC attached to CloudFront . . . . .          | 13        |
| 3.2 Network Evidence . . . . .                            | 13        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 3.2.1    | NE-1: VPC Endpoint created for DynamoDB . . . . .   | 14        |
| 3.2.2    | NE-2: Lambda has VpcConfig . . . . .                | 14        |
| 3.2.3    | NE-3: Route table has Endpoint route . . . . .      | 15        |
| 3.2.4    | NE-4: No NAT Gateway exists . . . . .               | 15        |
| 3.2.5    | NE-5: Lambda successfully calls DynamoDB . . . . .  | 16        |
| 3.3      | IAM evidence . . . . .                              | 16        |
| 3.3.1    | IM-1: Two separate IAM Roles . . . . .              | 16        |
| 3.3.2    | IM-2: Policy specifies exact resource ARN . . . . . | 17        |
| 3.3.3    | IM-3: Lambda attached to the correct Role . . . . . | 18        |
| 3.4      | Log evidence . . . . .                              | 18        |
| 3.4.1    | Successful request . . . . .                        | 19        |
| 3.4.2    | Error request . . . . .                             | 19        |
| <b>4</b> | <b>Monitoring Dashboard</b>                         | <b>21</b> |
| 4.1      | CloudWatch Dashboard . . . . .                      | 21        |
| 4.2      | Alarm . . . . .                                     | 22        |
| 4.3      | AWS Budget configuration page . . . . .             | 22        |
| <b>5</b> | <b>Cost Report</b>                                  | <b>24</b> |
| 5.1      | AWS Cost Explorer/Billing Dashboard . . . . .       | 24        |
| <b>6</b> | <b>Documentation</b>                                | <b>26</b> |
| 6.1      | Request Flow . . . . .                              | 26        |
| 6.1.1    | GET /tasks . . . . .                                | 26        |
| 6.1.2    | POST /tasks . . . . .                               | 28        |
| 6.1.3    | PUT /tasks/id . . . . .                             | 29        |
| 6.1.4    | DELETE /tasks/id . . . . .                          | 31        |
| 6.2      | Serverless auto-scaling . . . . .                   | 33        |

|          |                                                                                            |           |
|----------|--------------------------------------------------------------------------------------------|-----------|
| 6.2.1    | Lambda scales from 0 to N: . . . . .                                                       | 33        |
| 6.2.2    | API Gateway as a Native Load Balancer: . . . . .                                           | 33        |
| 6.2.3    | No Application Load Balancer is needed . . . . .                                           | 33        |
| 6.3      | CDN and OAC . . . . .                                                                      | 34        |
| 6.3.1    | CloudFront Caching . . . . .                                                               | 34        |
| 6.3.2    | Origin Access Control (OAC) . . . . .                                                      | 34        |
| 6.3.3    | Private S3 Bucket vs Static Website Hosting . . . . .                                      | 35        |
| 6.4      | VPC Endpoint . . . . .                                                                     | 35        |
| 6.4.1    | Reason Lambda inside a Private Subnet needs a VPC Endpoint to reach<br>DynamoDB: . . . . . | 35        |
| 6.4.2    | The Traffic Path: . . . . .                                                                | 36        |
| 6.4.3    | VPC Endpoint vs NAT Gateway . . . . .                                                      | 37        |
| <b>7</b> | <b>Source Code</b>                                                                         | <b>38</b> |

## LIST OF FIGURES

|             |                                              |    |
|-------------|----------------------------------------------|----|
| Figure 1.1  | GET /tasks . . . . .                         | 7  |
| Figure 1.2  | POST /tasks . . . . .                        | 7  |
| Figure 1.3  | PUT /tasks/id . . . . .                      | 8  |
| Figure 1.4  | DELETE /tasks/id . . . . .                   | 8  |
| Figure 1.5  | Database Explorer . . . . .                  | 9  |
| Figure 2.1  | Architecture Diagram . . . . .               | 10 |
| Figure 3.1  | SE-1 . . . . .                               | 11 |
| Figure 3.2  | SE-2 . . . . .                               | 12 |
| Figure 3.3  | SE-3 . . . . .                               | 12 |
| Figure 3.4  | Origins Access List . . . . .                | 13 |
| Figure 3.5  | Origins Access Setting . . . . .             | 13 |
| Figure 3.6  | NE-1 . . . . .                               | 14 |
| Figure 3.7  | NE-2 . . . . .                               | 14 |
| Figure 3.8  | NE-3 . . . . .                               | 15 |
| Figure 3.9  | NE-4 . . . . .                               | 15 |
| Figure 3.10 | NE-5 . . . . .                               | 16 |
| Figure 3.11 | IM-1 . . . . .                               | 16 |
| Figure 3.12 | Lambda Base Role's Policy . . . . .          | 17 |
| Figure 3.13 | Lambda Task Role's Policy . . . . .          | 17 |
| Figure 3.14 | Lambda Execution Role by Action . . . . .    | 18 |
| Figure 3.15 | Lambda Execution Role by Resources . . . . . | 18 |

|                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------|----|
| Figure 3.16 Log showing the REPORT line with StatusCode 200, Duration, Billed<br>Duration, and Memory Used . . . . . | 19 |
| Figure 3.17 Sending missing input . . . . .                                                                          | 19 |
| Figure 3.18 Log showing an ERROR entry or stack trace triggered by sending missing<br>input . . . . .                | 20 |
| Figure 4.1 CloudWatch Dashboard showing at least 5 widgets with live data . . . . .                                  | 21 |
| Figure 4.2 2 alarms created . . . . .                                                                                | 22 |
| Figure 4.3 AWS Budget configuration page . . . . .                                                                   | 22 |
| Figure 4.4 AWS Budget alert configuration page . . . . .                                                             | 23 |
| Figure 5.1 AWS Cost Explorer . . . . .                                                                               | 24 |
| Figure 5.2 Billing Dashboard . . . . .                                                                               | 25 |

# CHAPTER 1

## Web Application

### 1.1. Task Assignment and Completion

| Member               | Responsibilities                                                                                                                                           | Completed Rate |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Dang Truong Nguyen   | Frontend (HTML, CSS, JS) and Backend (Lambda CRUD APIs, DynamoDB integration); IAM Roles configuration and API testing                                     | 100%           |
| Nguyen Thanh Gia Bao | Networking setup: VPC, Private Subnets, VPC Endpoint for DynamoDB, Lambda VpcConfig, Route Table; collect NE evidence                                      | 100%           |
| Le Tuan Loc          | Deployment and Security: S3 private, CloudFront + OAC, API Gateway (CORS, throttling); Monitoring (CloudWatch, Alarms, Budget); collect SE and IM evidence | 100%           |

Table 1.1: Team Contribution

### 1.2. Frontend accessible via the CloudFront URL

- Serverless Application URL: <https://d14g6okcs1rawz.cloudfront.net/index.html>

### 1.3. Backend API

- Backend API accessible via the API gateway URL: <https://py29vzblk3.execute-api.ap-southeast-2.amazonaws.com/prod>

### 1.3.1. API GET /tasks testing with Postman

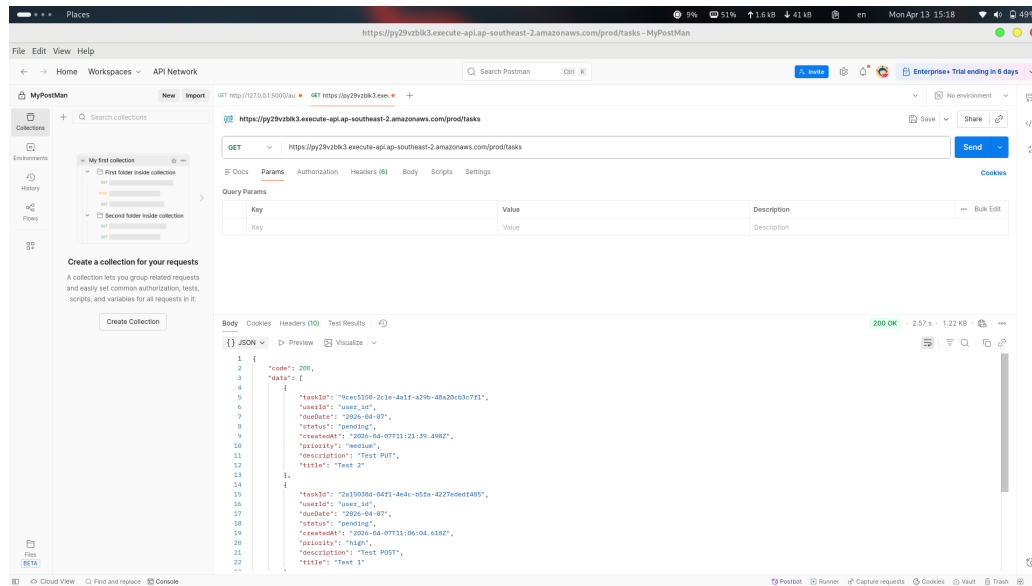


Figure 1.1: GET /tasks

### 1.3.2. API POST /tasks testing with Postman

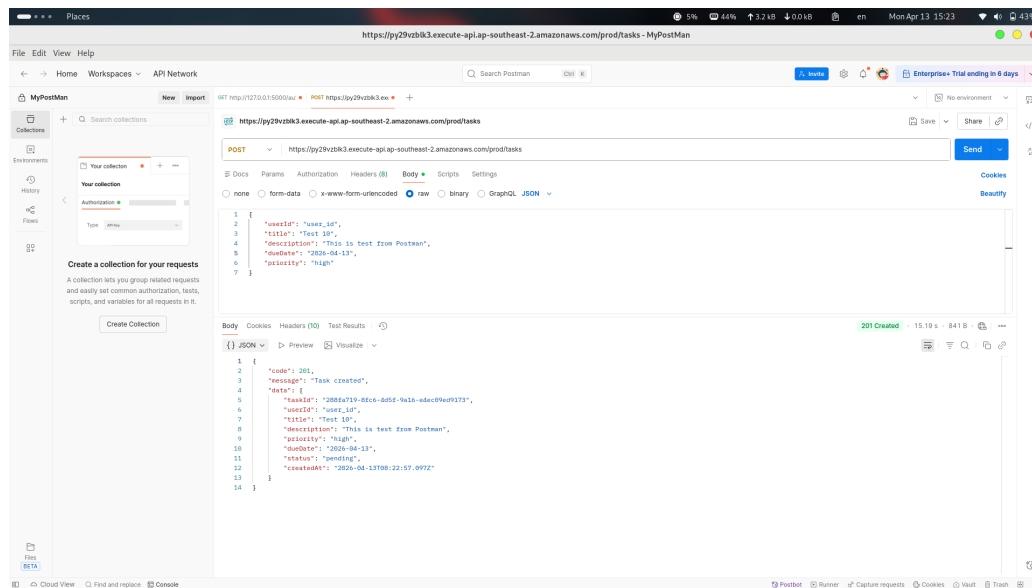


Figure 1.2: POST /tasks



### 1.3.3. API PUT /tasks/id testing with Postman

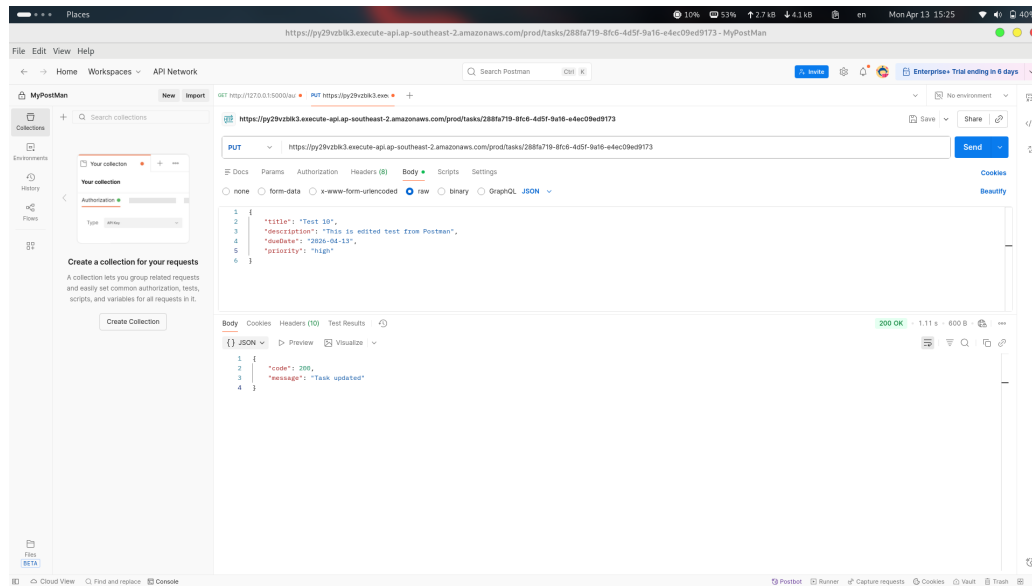


Figure 1.3: PUT /tasks/id

### 1.3.4. API DELETE /tasks/id testing with Postman

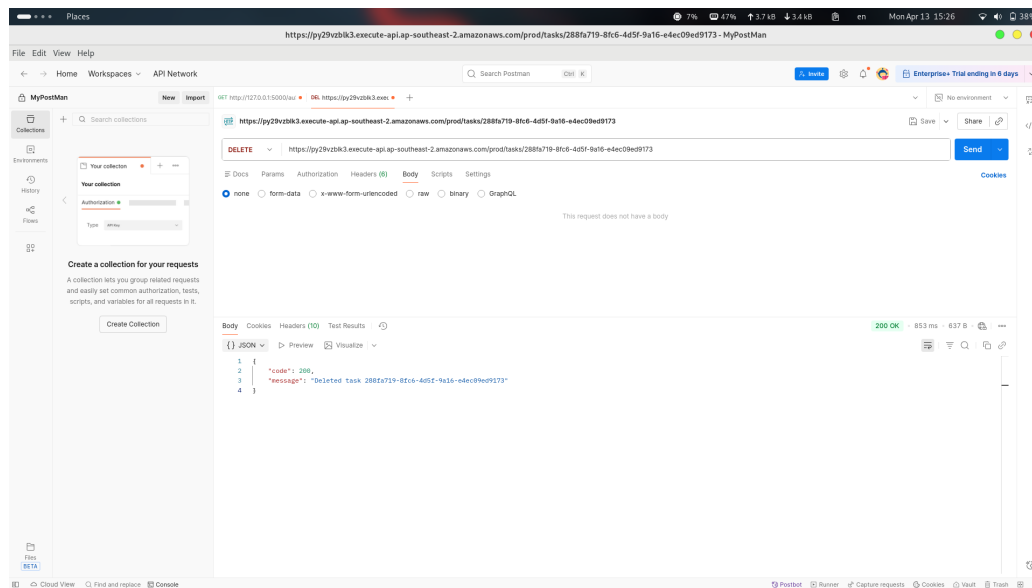


Figure 1.4: DELETE /tasks/id

## 1.4. Database Item

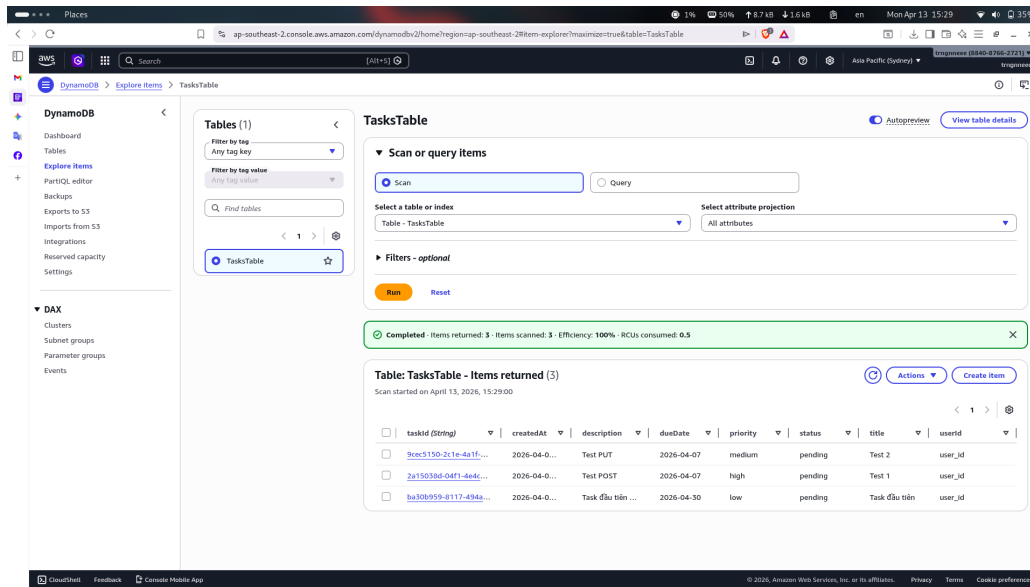


Figure 1.5: Database Explorer

# CHAPTER 2

## Architecture Diagram

### 2.1. Architecture Diagram

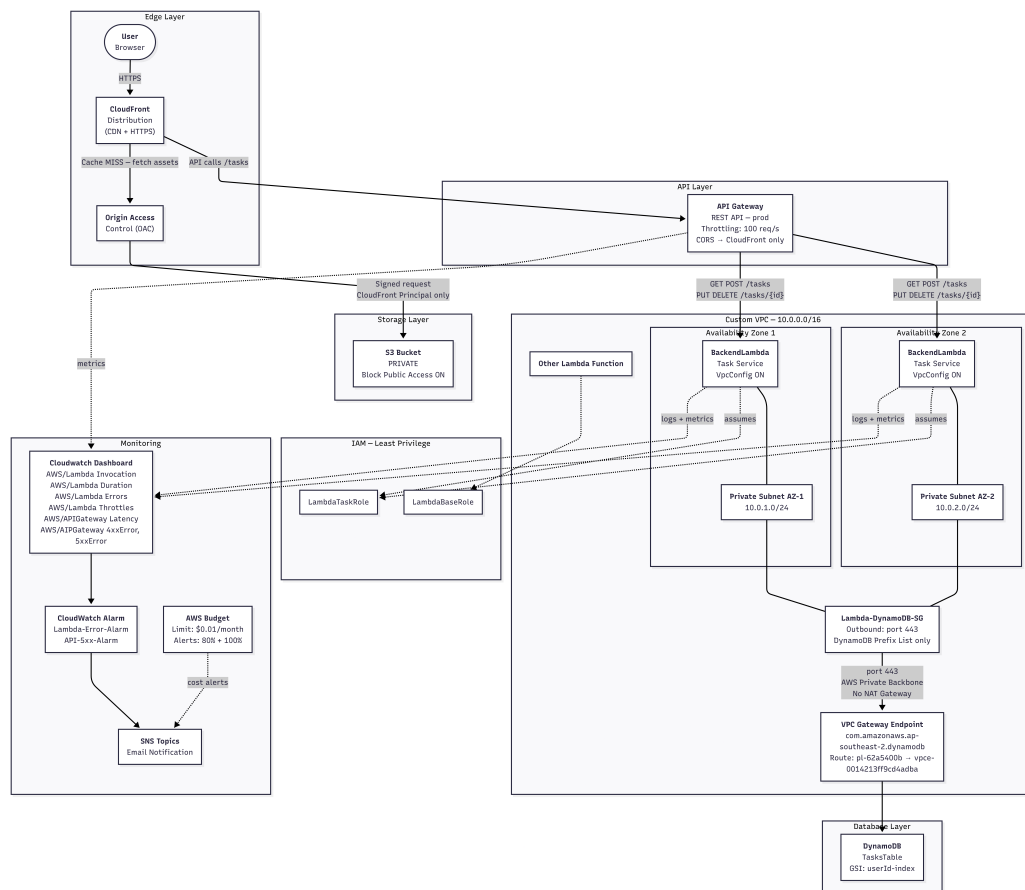


Figure 2.1: Architecture Diagram

# CHAPTER 3

## Security Evidence Package

### 3.1. Frontend Security Evidence

#### 3.1.1. SE-1: S3 Block Public Access enabled

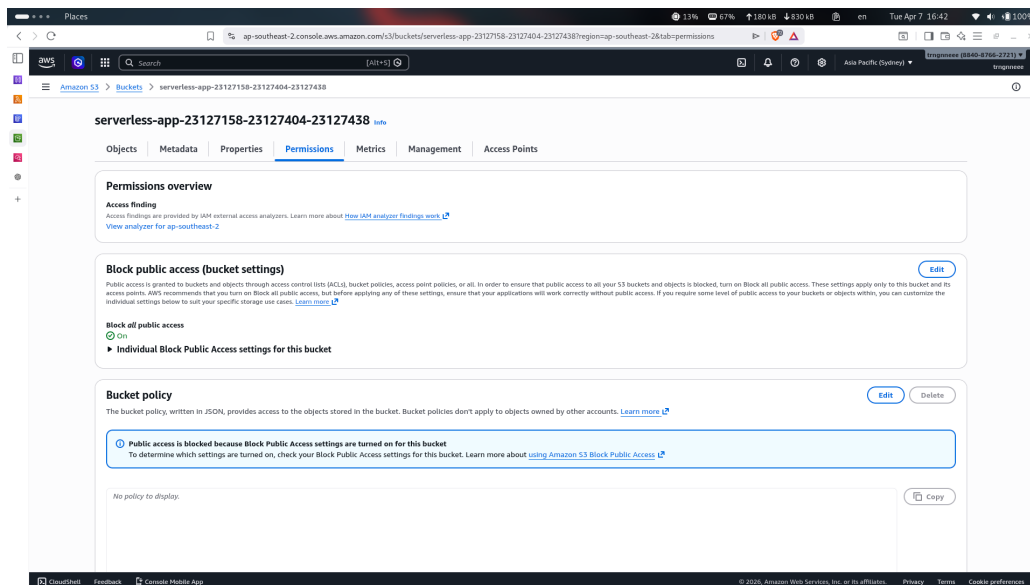


Figure 3.1: SE-1

### 3.1.2. SE-2: Direct S3 access rejected

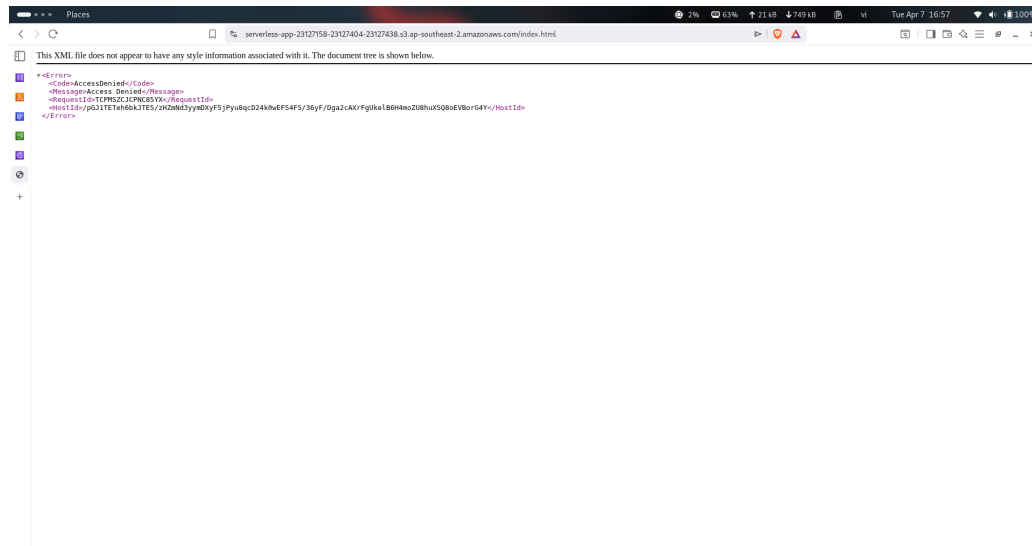


Figure 3.2: SE-2

### 3.1.3. SE-3: CloudFront access succeeds

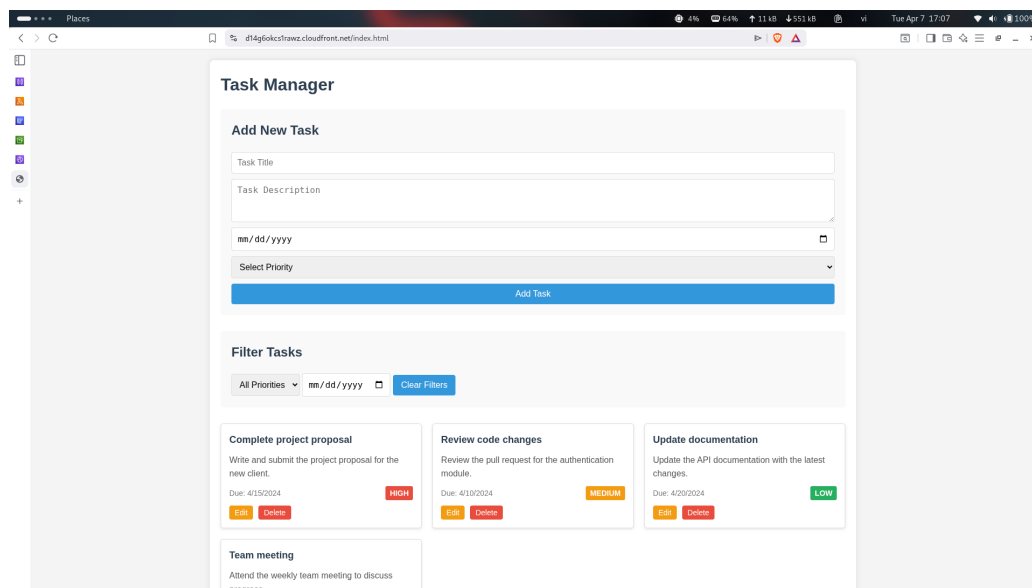


Figure 3.3: SE-3

### 3.1.4. SE-4: OAC attached to CloudFront

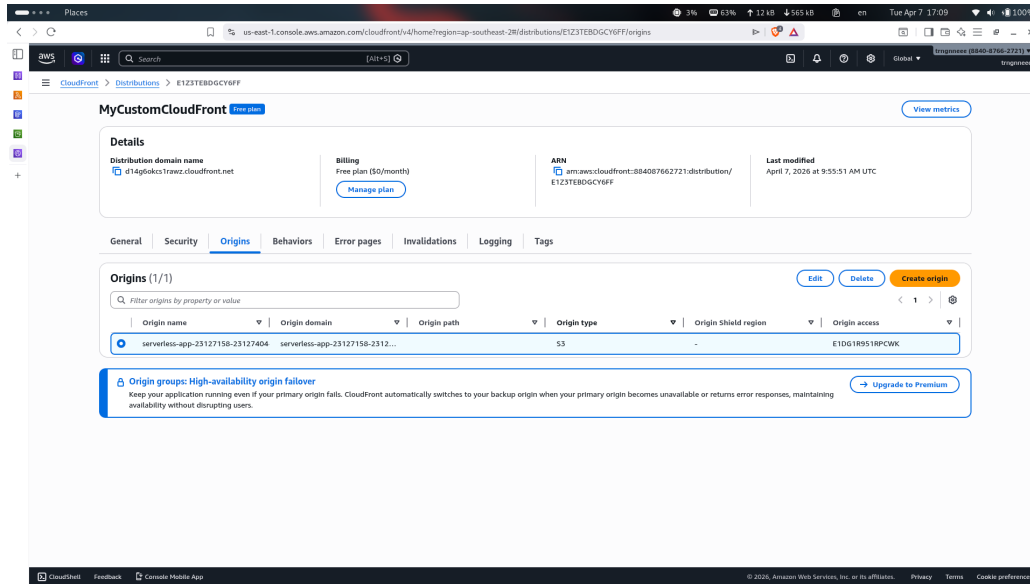


Figure 3.4: Origins Access List

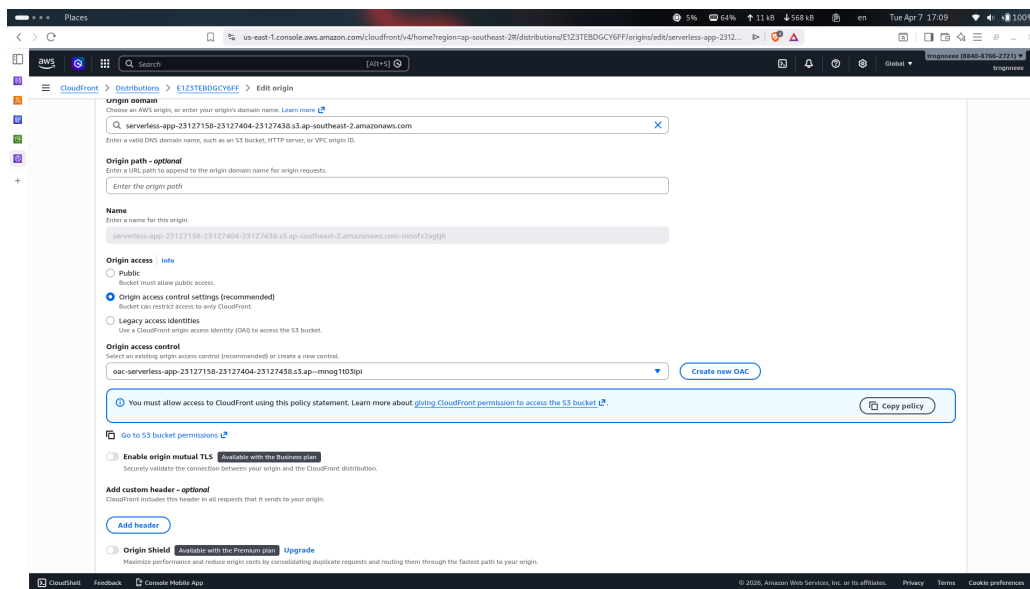


Figure 3.5: Origins Access Setting

## 3.2. Network Evidence

### 3.2.1. NE-1: VPC Endpoint created for DynamoDB

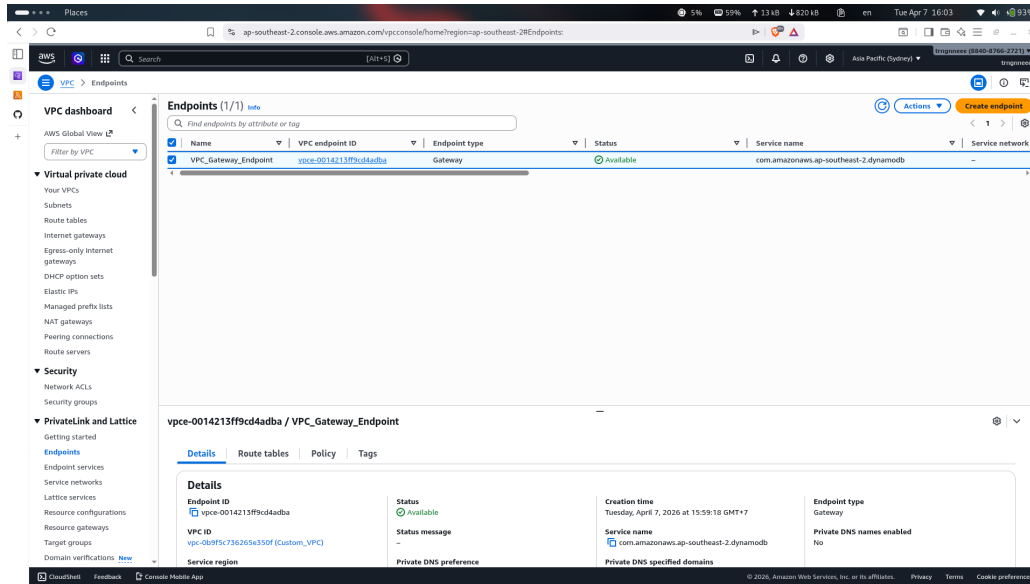


Figure 3.6: NE-1

### 3.2.2. NE-2: Lambda has VpcConfig

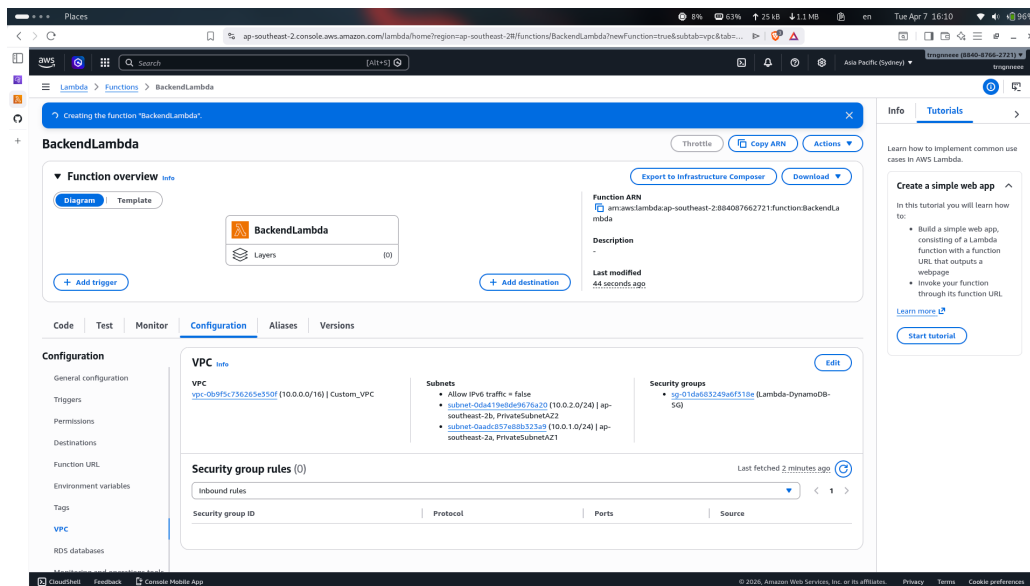


Figure 3.7: NE-2

### 3.2.3. NE-3: Route table has Endpoint route

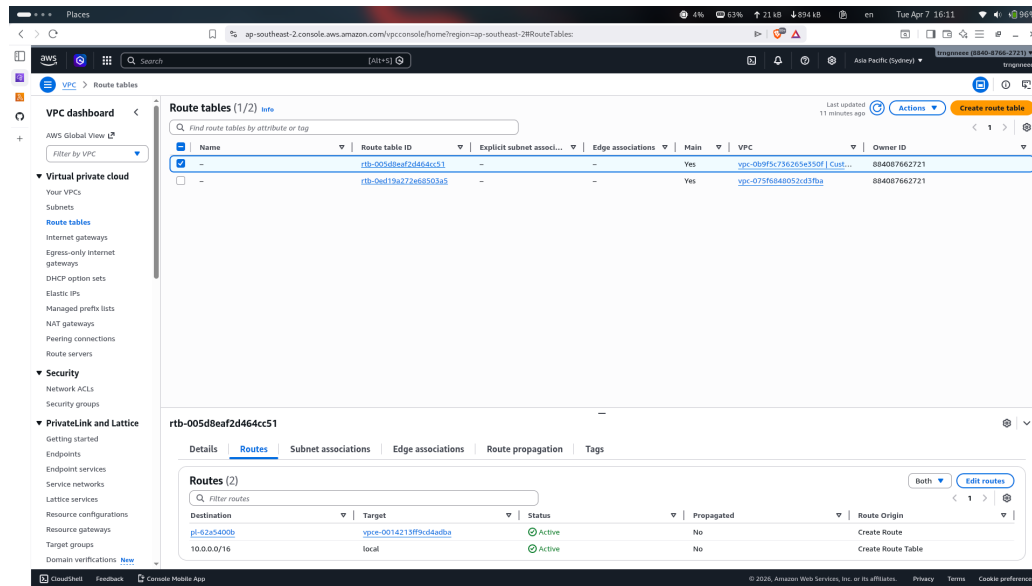


Figure 3.8: NE-3

### 3.2.4. NE-4: No NAT Gateway exists

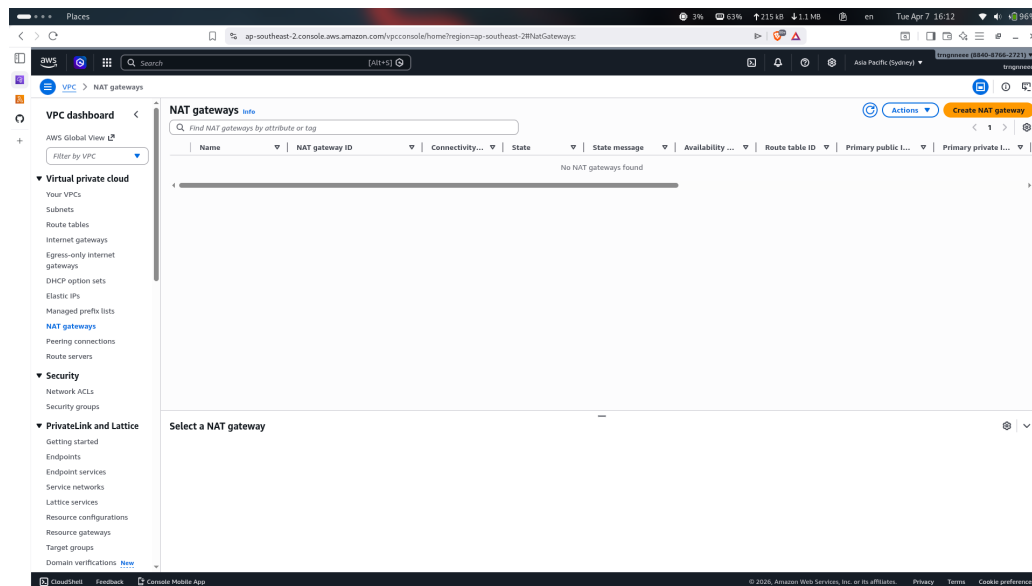


Figure 3.9: NE-4



### 3.2.5. NE-5: Lambda successfully calls DynamoDB

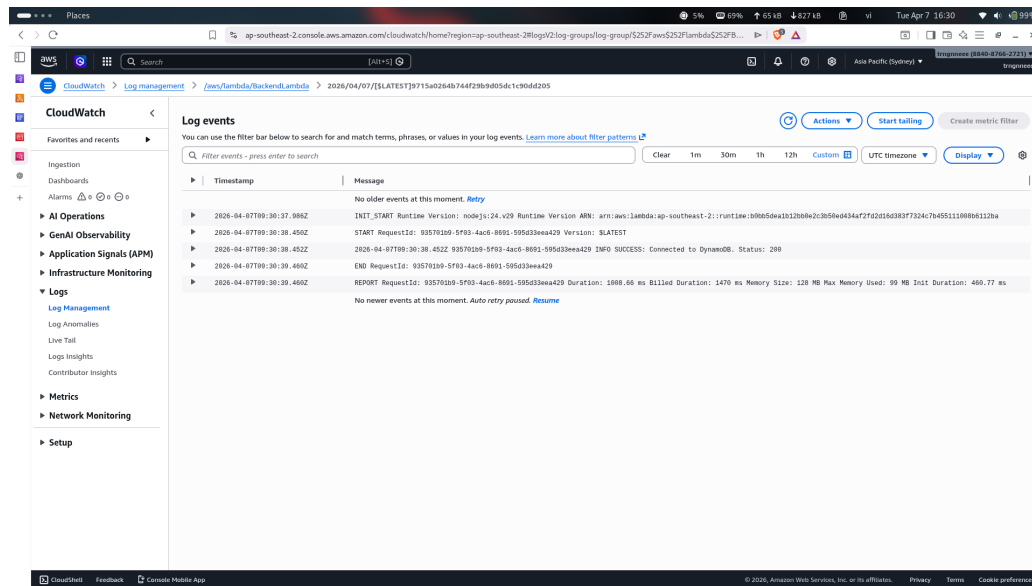


Figure 3.10: NE-5

## 3.3. IAM evidence

### 3.3.1. IM-1: Two separate IAM Roles

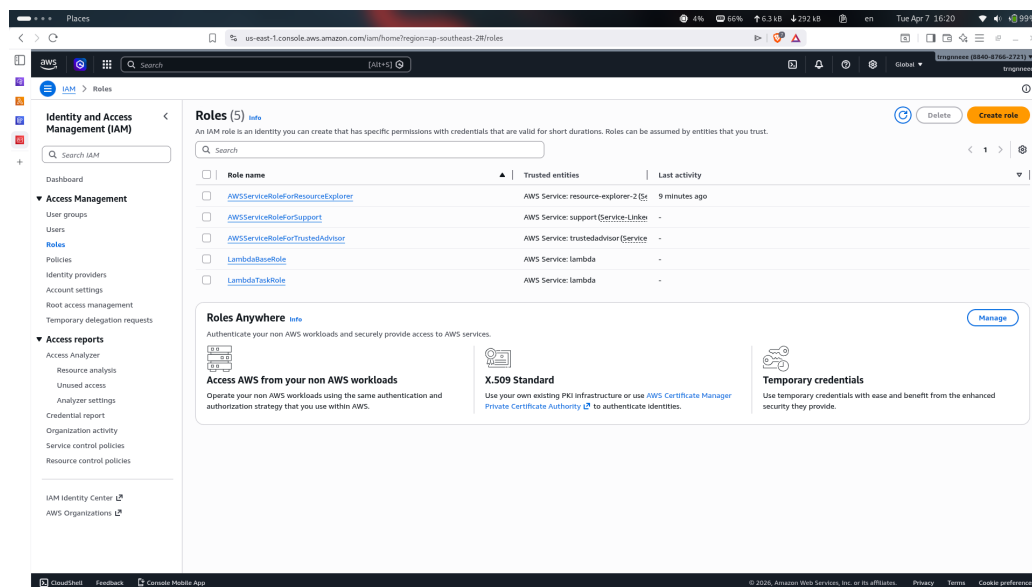


Figure 3.11: IM-1

### 3.3.2. IM-2: Policy specifies exact resource ARN

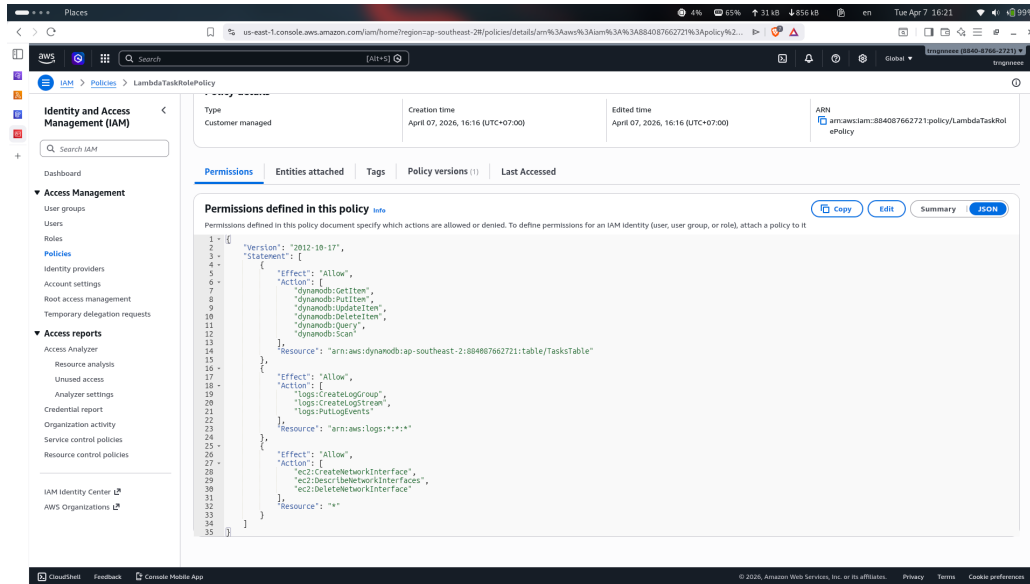


Figure 3.12: Lambda Base Role's Policy

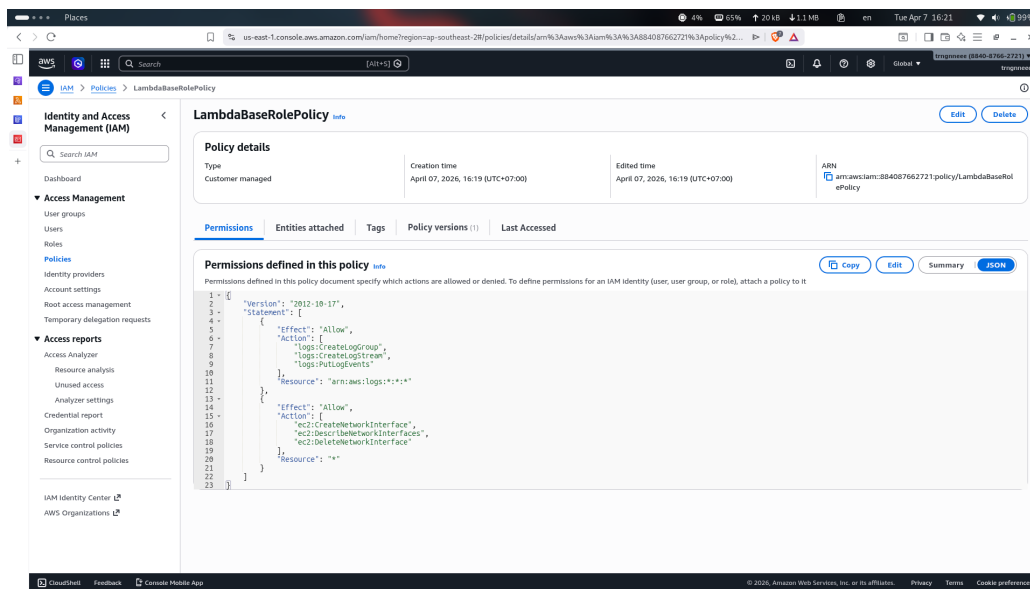


Figure 3.13: Lambda Task Role's Policy

### 3.3.3. IM-3: Lambda attached to the correct Role

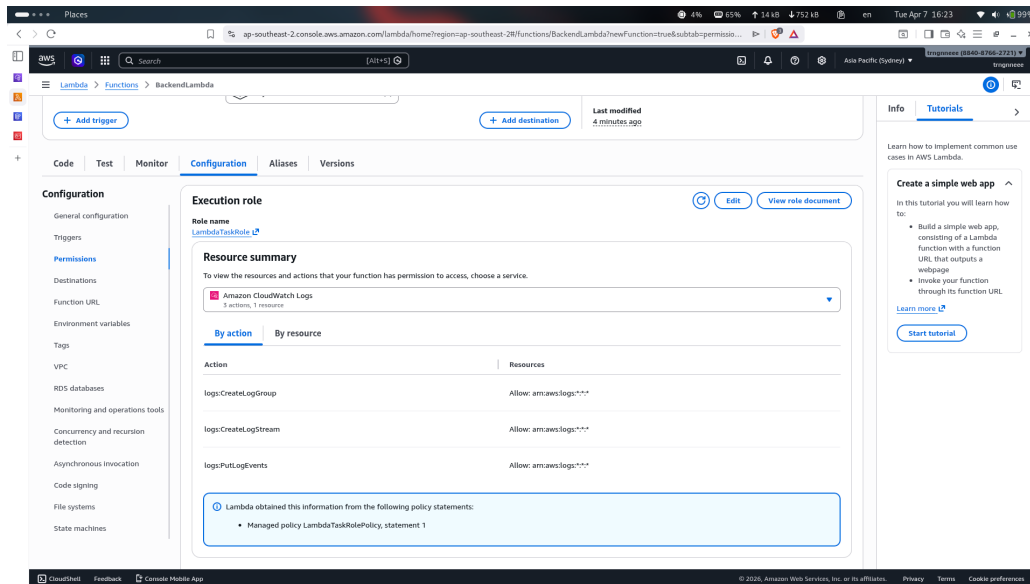


Figure 3.14: Lambda Execution Role by Action

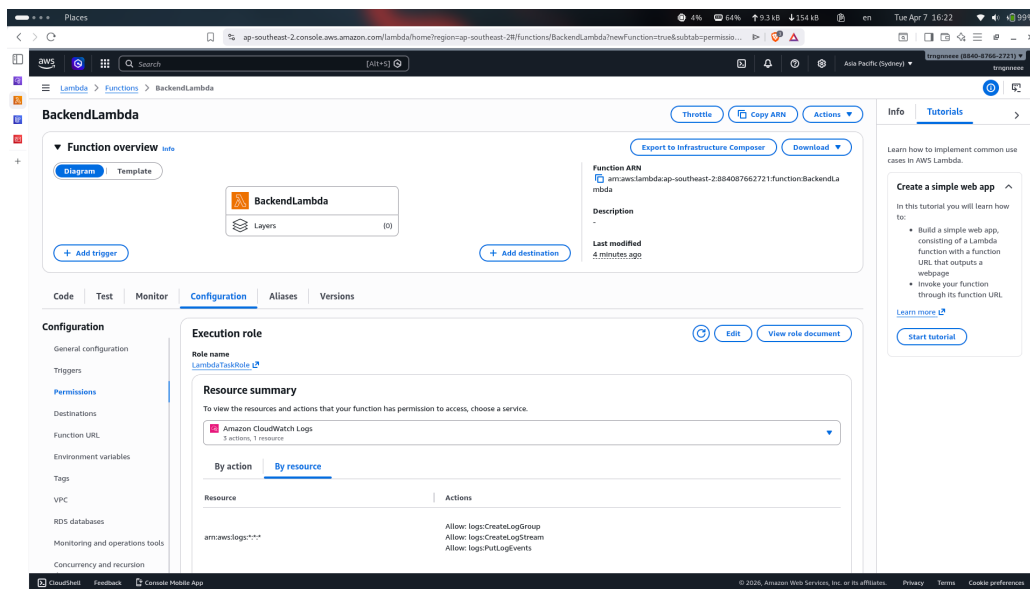


Figure 3.15: Lambda Execution Role by Resources

## 3.4. Log evidence



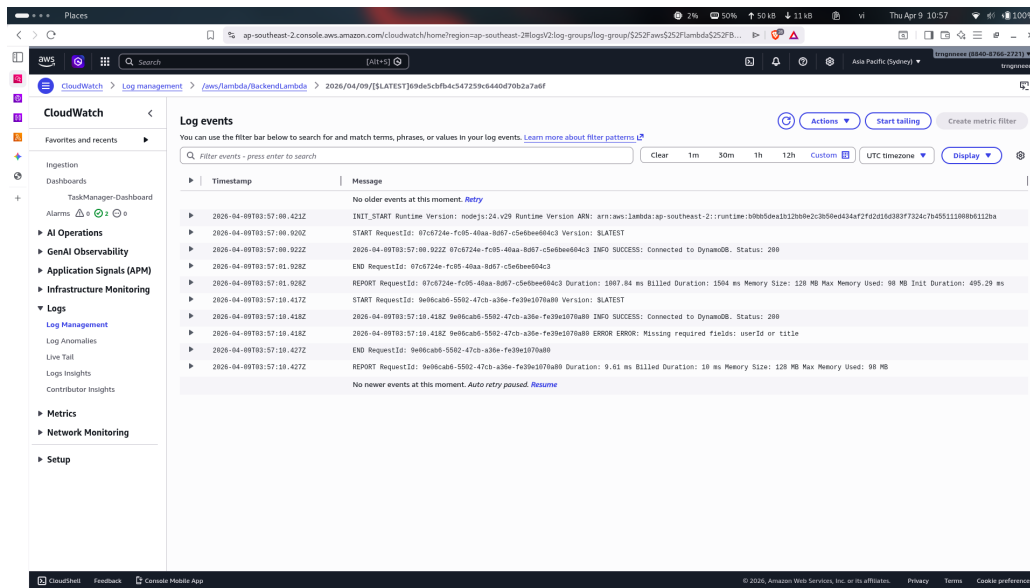


Figure 3.18: Log showing an ERROR entry or stack trace triggered by sending missing input

# CHAPTER 4

## Monitoring Dashboard

### 4.1. CloudWatch Dashboard

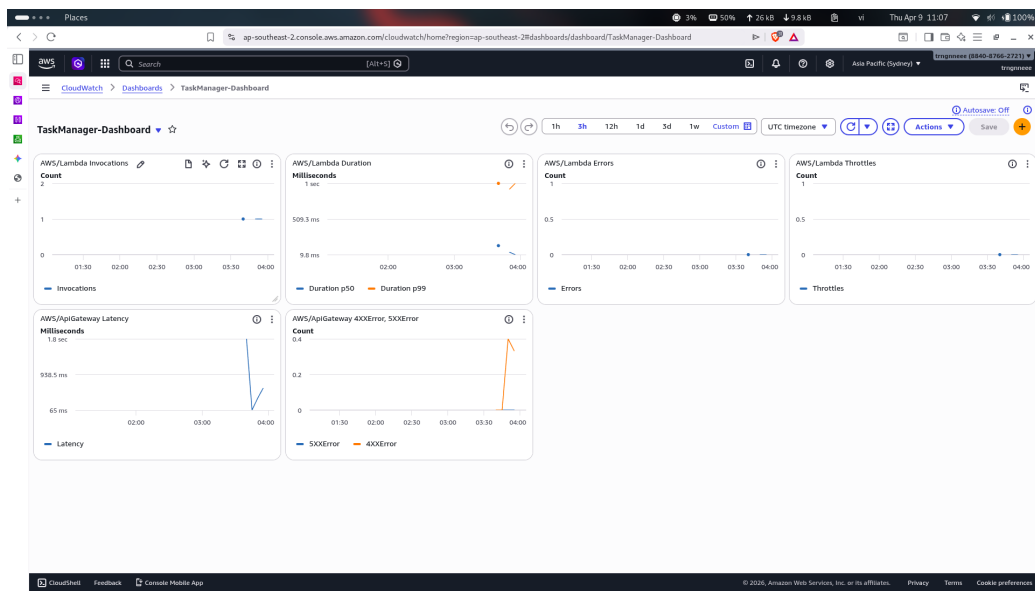


Figure 4.1: CloudWatch Dashboard showing at least 5 widgets with live data

## 4.2. Alarm

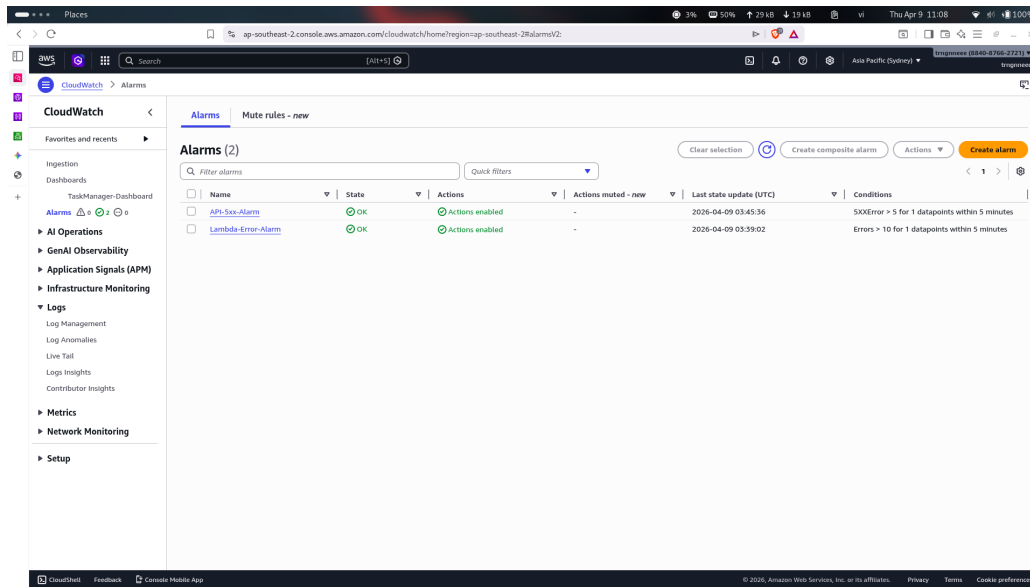


Figure 4.2: 2 alarms created

## 4.3. AWS Budget configuration page

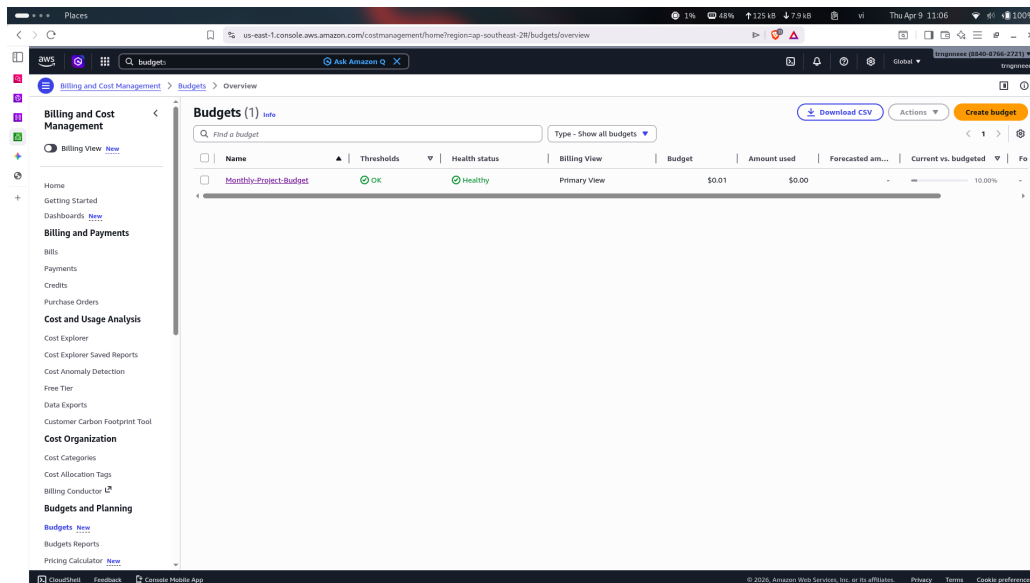


Figure 4.3: AWS Budget configuration page

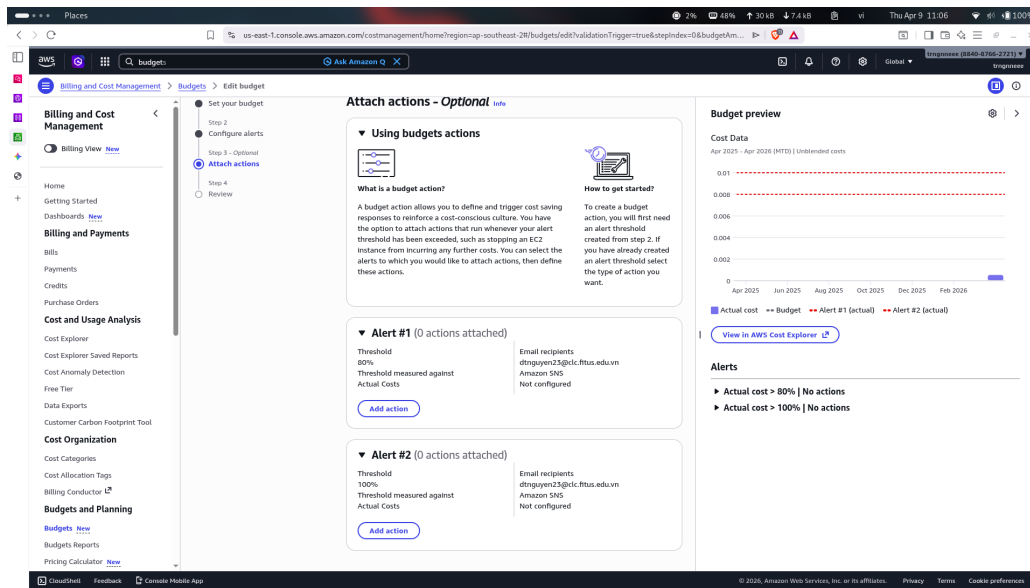


Figure 4.4: AWS Budget alert configuration page



# CHAPTER 5

## Cost Report

### 5.1. AWS Cost Explorer/Billing Dashboard

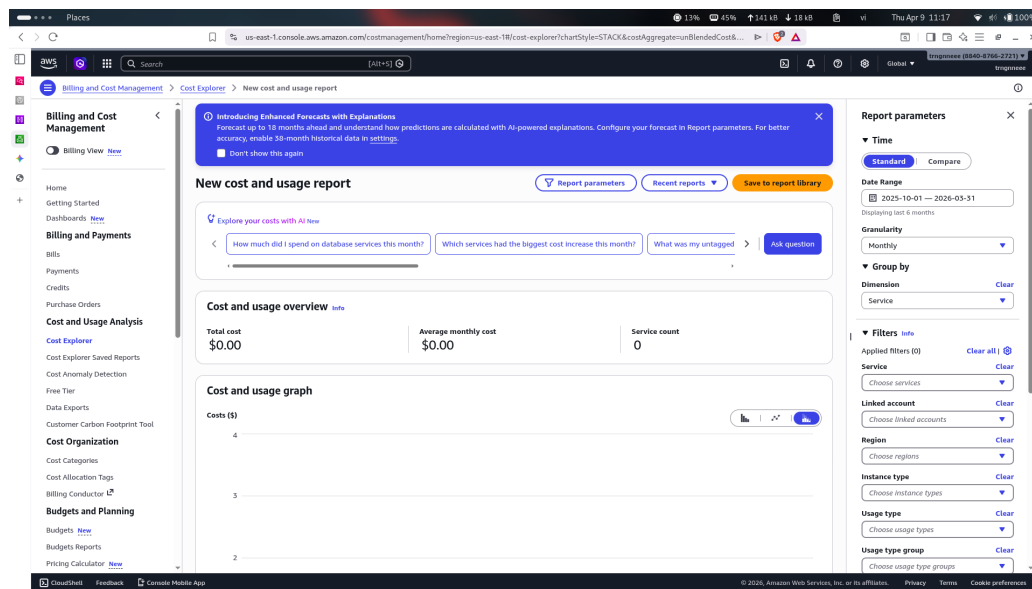


Figure 5.1: AWS Cost Explorer

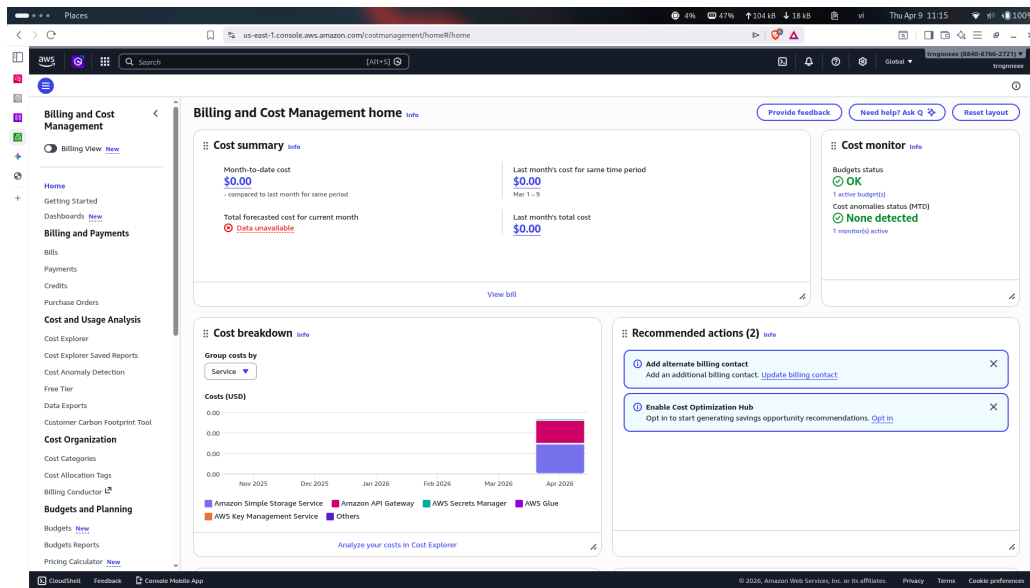


Figure 5.2: Billing Dashboard

# CHAPTER 6

## Documentation

### 6.1. Request Flow

#### 6.1.1. GET /tasks

- **Step 1: User access browser:**
  - The user accesses the application URL via their browser. The browser first requests the static frontend assets (HTML, CSS, JavaScript) from CloudFront.
- **Step 2: CloudFront and S3 (OAC):**
  - CloudFront retrieves the `index.html` and `script.js` from the private S3 bucket using Origin Access Control (OAC).
  - These assets are delivered to the browser with an HTTP 200 status.
- **Step 3: Frontend API Call:**
  - Once the page loads, the `script.js` executes a `fetch` request to the API Gateway endpoint.
  - The request is targeted at the `GET /tasks` path to retrieve all existing tasks.
- **Step 4: API Gateway Routing and Throttling:**
  - API Gateway receives the HTTPS request and validates the CORS policy against the specific CloudFront domain.
  - It applies Throttling rules (100 req/s rate and 50 burst) to limit request throughput.
- **Step 5: Lambda Trigger and VPC Entry:**
  - API Gateway triggers the Backend Lambda function, passing the request as an event.

- The function is initialized within the Custom VPC using `VpcConfig`, attaching to private subnets for network isolation.
- **Step 6: Network Isolation and Security Group:**
  - The Lambda function, residing in the private subnet, initiates a connection to DynamoDB.
  - The attached Security Group restricts outbound traffic to port 443 strictly for the DynamoDB Prefix List.
- **Step 7: Private Data Path via VPC Endpoint:**
  - The traffic does not traverse the public internet. Instead, the Route Table directs it through the VPC Gateway Endpoint.
  - This ensures all communication stays on the AWS private backbone for zero-cost and high security.
- **Step 8: DynamoDB Data Retrieval:**
  - The request reaches Amazon DynamoDB. Lambda executes a `Scan` or `Query` (via GSI) operation on the `TasksTable`.
  - Access is authorized by a dedicated IAM TaskRole using the Least Privilege principle.
- **Step 9: Observability and Logging:**
  - Lambda processes the retrieved data and logs execution details (StatusCode 200, Duration, Memory Used) to CloudWatch Logs.
  - Real-time metrics are updated on the TaskManager-Dashboard.
- **Step 10: Final Response:**
  - Lambda returns the JSON payload and required CORS headers to API Gateway.
  - API Gateway forwards the response to the browser, which renders the task list on the responsive interface.

### 6.1.2. POST /tasks

- **Step 1: User interaction in Browser:**

- The user fills out the "Add New Task" form and clicks the submit button.
- The browser's JavaScript captures the input data (title, description, priority, dueDate) and prepares a JSON payload.

- **Step 2: Preflight CORS Request (OPTIONS):**

- Since a POST request with JSON content is a non-simple request, the browser first sends an OPTIONS request to API Gateway.
- API Gateway validates the Origin against the CloudFront domain and returns the allowed methods and headers.

- **Step 3: Frontend API Call:**

- After a successful preflight, the `script.js` sends the actual POST request with the JSON body to the API Gateway endpoint.

- **Step 4: API Gateway Routing and Throttling:**

- API Gateway receives the HTTPS POST request, validates the domain, and applies throttling limits (100 req/s rate and 50 burst).
- It ensures the request volume does not exceed the Lambda Reserved Concurrency of 50.

- **Step 5: Lambda Trigger and VPC Entry:**

- API Gateway triggers the Lambda function, passing the task data in the event body.
- The function is initialized within the Custom VPC using its `VpcConfig`, attaching to private subnets across two AZs for High Availability.

- **Step 6: Network Isolation and Security Group:**

- The Lambda function operates inside the private network without a NAT Gateway.

- The Security Group allows outbound traffic on port 443 strictly to the DynamoDB Prefix List only.
- **Step 7: Private Data Path via VPC Endpoint:**
  - The request is routed through the VPC Gateway Endpoint for DynamoDB.
  - This ensures the task data travels entirely over the AWS private backbone, never traversing the public internet.
- **Step 8: DynamoDB PutItem Operation:**
  - Lambda generates a unique `taskId` (UUID) and a `createdAt` timestamp.
  - It executes the `PutItem` command to store the task in the `TasksTable` using the permissions granted by the dedicated IAM TaskRole.
- **Step 9: Observability and Logging:**
  - Lambda logs the successful creation or any validation errors (e.g., missing required fields) to CloudWatch Logs.
  - Metrics such as Invocations and Duration are sent to the TaskManager-Dashboard.
- **Step 10: Final Response:**
  - Lambda returns a 201 Created status code along with the new task object and required CORS headers.
  - API Gateway forwards the response to the browser, which updates the UI to display the new task.

### 6.1.3. PUT /tasks/id

- **Step 1: User interaction in Browser:**
  - The user modifies a task (e.g., changing status to "done" or updating priority) and clicks the update button.
  - The JavaScript code identifies the `taskId` and prepares a JSON object containing only the updated fields.

- **Step 2: Preflight CORS Request (OPTIONS):**

- The browser sends an OPTIONS request to the API Gateway to verify permissions for the PUT method.
- API Gateway confirms the CloudFront origin and returns the necessary Access-Control headers.

- **Step 3: Frontend API Call:**

- The frontend executes a `fetch` request to the specific resource path: `PUT /tasks/taskId`.

- **Step 4: API Gateway Routing and Throttling:**

- API Gateway receives the HTTPS request and matches it to the PUT method of the `/tasks/id` resource.
- It applies rate limiting (100 req/s) and burst limits (50) to protect the backend.

- **Step 5: Lambda Trigger and VPC Entry:**

- The Backend Lambda function is triggered with the `taskId` passed as a path parameter.
- The function operates within the Custom VPC via its `VpcConfig`, ensuring all traffic is isolated from the public internet.

- **Step 6: Network Isolation and Security Group:**

- Lambda initiates a secure connection from the private subnet.
- The Security Group restricts outbound communication to port 443, directed only towards the DynamoDB Prefix List.

- **Step 7: Private Data Path via VPC Endpoint:**

- The update request is routed through the VPC Gateway Endpoint for DynamoDB.
- This ensures the task data stays within the AWS private backbone, maintaining zero-cost and defense-in-depth security.

- **Step 8: DynamoDB UpdateItem Operation:**

- Lambda executes the `UpdateItem` command on the `TasksTable`.
- It updates specific attributes using an Expression Attribute map, authorized by the IAM TaskRole with Least Privilege permissions.
- **Step 9: Observability and Monitoring:**
  - Lambda logs the outcome to CloudWatch Logs, including the REPORT line with Duration and Memory metrics.
  - The CloudWatch Dashboard reflects the updated metrics for Latency and Invocations.
- **Step 10: Final Response:**
  - Lambda returns a 200 OK status and the updated task data along with CORS headers.
  - API Gateway forwards the response to the browser, which refreshes the UI to show the updated task status.

#### 6.1.4. DELETE /tasks/id

- **Step 1: User interaction in Browser:**
  - The user clicks the "Delete" icon or button next to a specific task.
  - The browser's JavaScript identifies the unique `taskId` associated with that task.
- **Step 2: Preflight CORS Request (OPTIONS):**
  - The browser automatically sends an OPTIONS request to API Gateway to confirm that the DELETE method is permitted from the CloudFront origin.
  - API Gateway responds with the necessary CORS headers.
- **Step 3: Frontend API Call:**
  - After a successful preflight, the JavaScript executes a `fetch` request to the API Gateway endpoint with the DELETE method, targeting the specific resource path: `/tasks/taskId`.
- **Step 4: API Gateway Routing and Throttling:**



- API Gateway receives the request and applies Throttling rules, including a rate of 100 req/s and a burst limit of 50.
- It routes the request to the Backend Lambda function.
- **Step 5: Lambda Trigger and VPC Entry:**
  - The Lambda function is triggered and initialized within the Custom VPC using `VpcConfig`.
  - It connects to two private subnets to ensure High Availability.
- **Step 6: Network Isolation and Security Group:**
  - The Lambda function, isolated in a private subnet, initiates communication with DynamoDB.
  - The Security Group restricts outbound traffic to port 443 only for the DynamoDB Prefix List.
- **Step 7: Private Data Path via VPC Endpoint:**
  - The traffic is directed through the VPC Gateway Endpoint, staying entirely on the AWS private backbone.
  - This avoids any exposure to the public internet and prevents NAT Gateway costs.
- **Step 8: DynamoDB DeleteItem Operation:**
  - Lambda executes the `DeleteItem` command on the `TasksTable` using the provided `taskId`.
  - The operation is authorized by the IAM TaskRole following the Least Privilege principle.
- **Step 9: Observability and Monitoring:**
  - Lambda logs the completion of the deletion to CloudWatch Logs, showing the REPORT line with Duration and Memory metrics.
  - The execution is tracked by the TaskManager-Dashboard widgets.
- **Step 10: Final Response:**
  - Lambda returns a 200 OK status code and a confirmation message in JSON format.
  - API Gateway forwards the response to the browser, which then removes the task from the display.

## 6.2. Serverless auto-scaling

### 6.2.1. Lambda scales from 0 to N:

- **Scale from 0:** When there are no incoming requests, the Lambda function has zero active instances, resulting in no cost. When a request arrives, AWS automatically provisions a new instance of the Lambda function to handle it, allowing for seamless scaling without any manual intervention.
- **Scale to N:** As the number of incoming requests increases, AWS continues to create additional instances of the Lambda function as needed, up to the configured limits (e.g., Reserved Concurrency). This allows the application to handle varying levels of traffic without performance degradation, while still maintaining a cost-effective architecture since you only pay for the compute time used by each invocation.

### 6.2.2. API Gateway as a Native Load Balancer:

- In a serverless architecture, API Gateway functions as the entry point and distributor for all traffic. It automatically routes incoming requests to the appropriate Lambda function based on the defined API routes and methods.
  - **Traffic Distribution:** It receives HTTPS requests from the internet and automatically routes them to the available Lambda execution environments, effectively balancing the load across multiple instances of the function as they scale up.
  - **Throttling and Rate Limiting:** API Gateway also provides built-in throttling and rate limiting capabilities to prevent abuse and ensure fair usage of the API.
  - **Native Integration:** Since API Gateway is designed to work seamlessly with Lambda, it eliminates the need for a traditional load balancer, simplifying the architecture and reducing operational overhead while still providing high availability and scalability.

### 6.2.3. No Application Load Balancer is needed

- An Application Load Balancer is typically used to distribute traffic across a fleet of virtual machines (EC2) or containers. It is not required for this project because:

- **Serverless Native Integration:** API Gateway is designed to work directly with Lambda functions, providing built-in routing and load distribution without the need for an additional load balancer.
- **Cost and Complexity:** Introducing an Application Load Balancer would add unnecessary cost and complexity to the architecture, as it would require additional resources to manage and maintain.
- **Scalability:** Lambda functions automatically scale in response to incoming traffic, and API Gateway can handle the routing without any bottlenecks, making an Application Load Balancer redundant in this serverless setup.

## 6.3. CDN and OAC

### 6.3.1. CloudFront Caching

- CloudFront speeds up the delivery of the application by storing copies of the static assets (HTML, CSS, JavaScript) in edge locations around the world.
  - **Cache HIT:** Occurs when user requests a file and a valid version of that file is already stored in the edge location closest to the user. The file is delivered directly from the cache, resulting in faster load times and reduced latency.
  - **Cache MISS:** Happens when the requested file is not found at the edge location or the cached version has expired. CloudFront must then go back to the S3 Bucket to retrieve the file, send it to the user, and store a copy in its cache for future requests.
- To reset the cache, we can create a new Invalidation in CloudFront, specifying the paths to the assets that need to be refreshed (e.g., `/index.html`, `/script.js`). This forces CloudFront to fetch the latest version from the S3 bucket on the next request.

### 6.3.2. Origin Access Control (OAC)

- **Definition:** Origin Access Control (OAC) is a security feature in CloudFront that allows the CDN to authenticate and securely communicate with an S3 bucket.

- **Reason OAC is required:** OAC is necessary because it ensures that only CloudFront distribution can read the objects inside your S3 bucket. It prevents users from bypassing the CDN to access the files directly, which is critical for maintaining the "Zero-Cost Secure Architecture" of this project.

### 6.3.3. Private S3 Bucket vs Static Website Hosting

- **Private Bucket:** The bucket must remain private with all "Block Public Access" options enabled to prevent unauthorized internet users from scanning or downloading your source code directly.
- **No Static Website Hosting:** Public S3 Bucket with Static Website Hosting enabled would allow anyone to access the files directly via the S3 URL, which is a security risk because it bypasses CloudFront's security and caching, violating the "defense-in-depth" architecture required for this assignment. By keeping the bucket private and using OAC, we ensure that all access goes through CloudFront, which can enforce security policies and provide better performance.
- **Access Path:** All traffic must flow through CloudFront, which retrieves the assets from the private S3 bucket using OAC. This setup ensures that the application benefits from CloudFront's caching and security features while keeping the S3 bucket secure and inaccessible to direct public access, and the S3 only serves as an origin for CloudFront, not a public endpoint.

## 6.4. VPC Endpoint

### 6.4.1. Reason Lambda inside a Private Subnet needs a VPC Endpoint to reach DynamoDB:

- **Default Isolation:** When a Lambda function is placed inside a private subnet, it does not have direct access to the internet. This means it does not have a public IP address or a path to the internet, so it cannot reach public AWS services like DynamoDB without a VPC Endpoint or a NAT Gateway.

- **Public Service Nature:** AWS DynamoDB is a public service that typically requires internet access to communicate with it. Without a VPC Endpoint, the Lambda function would need to route traffic through a NAT Gateway to access DynamoDB, which would incur additional costs and potentially introduce latency.
- **Bridge:** Without a VPC Endpoint, the Lambda function would be effectively isolated from DynamoDB, making it impossible to perform any database operations. The VPC Endpoint serves as a bridge that allows the Lambda function to securely and privately communicate with DynamoDB without exposing it to the public internet, thus maintaining the security and cost-efficiency of the architecture.

#### 6.4.2. The Traffic Path:

- **Internal Routing:** When Lambda sends a request to DynamoDB, the VPC Route Table identifies the destination (the DynamoDB Prefix List) and directs the traffic toward the VPC Gateway Endpoint.
- **Private Backbone:** The traffic travels through the AWS private backbone network, ensuring that it does not traverse the public internet at any point. This provides enhanced security and eliminates data transfer costs associated with NAT Gateway usage.
- **No Internet Traversal:** Because the request never leaves the AWS global network, it is not exposed to the risks of the public web, and it bypasses the need for internet-facing gateways. This design adheres to the "Zero-Cost Secure Architecture" principle by avoiding unnecessary components and ensuring all communication remains private and secure within AWS.
- **Direct Access:** The VPC Endpoint allows the Lambda function to directly access DynamoDB without needing to route through a NAT Gateway, which would be required if the Lambda function were in a public subnet. This direct access is crucial for maintaining low latency and high performance while keeping costs down.
- **Security Group Enforcement:** The Security Group attached to the Lambda function ensures that only traffic destined for DynamoDB (port 443 to the DynamoDB Prefix List) is allowed, further securing the communication path and preventing any unauthorized access attempts.

- **Outcome:** The result is a secure, efficient, and cost-effective communication path between the Lambda function and DynamoDB, fully leveraging AWS's private network infrastructure while adhering to best practices for security and architecture design.

#### 6.4.3. VPC Endpoint vs NAT Gateway

| Feature    | VPC Gateway Endpoint (Used in Project)                                     | NAT Gateway (Prohibited)                                                   |
|------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Cost       | \$0 / month (Free for DynamoDB)                                            | ≈ \$32 / month base charge                                                 |
| Security   | <b>Higher:</b> Traffic is strictly private and never touches the internet. | <b>Lower:</b> Traffic traverses the NAT Gateway to reach public endpoints. |
| Complexity | Simple configuration via Route Tables.                                     | Requires Public Subnets and an Internet Gateway.                           |
| Efficiency | Direct access to the AWS private backbone.                                 | Adds a networking hop through an external gateway.                         |

Table 6.1: Comparison between VPC Gateway Endpoint and NAT Gateway

## CHAPTER 7

### Source Code

- Github URL: [https://github.com/trngnneee/AWS\\_Serverless\\_Application](https://github.com/trngnneee/AWS_Serverless_Application)